

METHODEN UND WERKZEUGE DER SOFTWAREENTWICKLUNG

PROJEKTARBEIT

Just-in-Time-Privilege-Elevation

Ein Modul für zeitlich befristete,
zwei-faktor-gesicherte Rechte-Erhöhung

Autor: Lukas Cvitanović

Matrikelnr.: _____

Kurs: _____

Betreuer: _____

27. Mai 2026

Inhaltsverzeichnis

1	Einleitung und Motivation	2
2	Anforderungen	2
2.1	Funktionale Anforderungen	2
2.2	Nicht-funktionale Anforderungen	2
3	Stand der Technik	3
4	Konzept und Architektur	3
4.1	Überblick	3
4.2	Lebenszyklus eines Grants	3
4.3	Ablauf	3
4.4	Datenmodell	6
5	Sicherheitsanalyse	6
5.1	Vertrauensgrenzen	6
5.2	Bedrohungsanalyse nach STRIDE	7
5.3	Fail-closed als durchgängiges Prinzip	7
6	Implementierung	8
6.1	Modul-Struktur	8
6.2	Die Schnittstellen (Ports)	8
6.3	Der sicherheitskritische Pfad: Confirm	8
6.4	TOTP nach RFC 6238	9
7	Evaluation	9
7.1	Tests und Abdeckung	9
7.2	Durch Tests aufgedeckte Defekte	9
7.3	Limitierungen	9
8	Fazit und Ausblick	9

1 Einleitung und Motivation

Administrative Berechtigungen werden in vielen Systemen dauerhaft vergeben: Ein Konto besitzt erhöhte Rechte unabhängig davon, ob diese im Moment tatsächlich benötigt werden. Diese Praxis widerspricht dem Least-Privilege-Prinzip und vergrößert den potenziellen Schaden einer Kontoübernahme erheblich, da ein kompromittiertes Administratorkonto jederzeit voll handlungsfähig ist. Zugleich erschwert sie die Nachvollziehbarkeit: Aus einer dauerhaften Rechtevergabe lässt sich nicht ablesen, wann und wozu ein Recht tatsächlich ausgeübt wurde.

Als Antwort darauf hat sich *Just-in-Time*-Zugriff (JIT) als Stand der Technik etabliert. Erhöhte Rechte werden nur für den konkreten Bedarfsmoment gewährt und laufen anschließend automatisch ab. Der Zeitraum, in dem ein kompromittiertes Konto Schaden anrichten kann, wird damit auf wenige Minuten begrenzt, und jede Erhöhung wird zu einem protokollierbaren Ereignis.

Die vorliegende Arbeit entwirft und implementiert ein eigenständiges, wiederverwendbares Modul, das JIT-Privilege-Elevation als Bibliothek bereitstellt. Das Modul ist bewusst vom Trägersystem entkoppelt und wird beispielhaft in eine bestehende, selbst entwickelte PaaS-Umgebung integriert. Nach der Anforderungsanalyse (Abschnitt 2) und der Einordnung in den Stand der Technik (Abschnitt 3) beschreibt Abschnitt 4 das Konzept, Abschnitt 5 die Sicherheitsanalyse nach STRIDE, Abschnitt 6 die Implementierung und Abschnitt 7 die Evaluation.

2 Anforderungen

2.1 Funktionale Anforderungen

- Ein Benutzer kann eine zeitlich befristete Rechte-Erhöhung *beantragen*.
- Die Erhöhung wird erst nach erneuter Zwei-Faktor-Bestätigung (Step-Up-Authentication) wirksam.
- Aktive Erhöhungen lassen sich vorzeitig widerrufen und laufen andernfalls automatisch ab.
- Jeder Schritt wird manipulationssicher protokolliert.

2.2 Nicht-funktionale Anforderungen

Fail-closed Im Zweifel oder bei jedem Fehler wird kein Recht gewährt.

Unabhängigkeit Der Kern ist vom Trägersystem entkoppelt (Ports-and-Adapters).

Nachvollziehbarkeit Lückenloser, integritätsgesicherter Audit-Trail.

Korrektheit Verifiziert durch automatisierte Tests inklusive Nebenläufigkeit.

3 Stand der Technik

Verfahren zur kontrollierten Rechteerhöhung sind etabliert; sie unterscheiden sich vor allem in Granularität und Betriebsaufwand. *HashiCorp Vault* stellt über seine SSH-Secrets-Engine kurzlebige, signierte SSH-Zertifikate aus und verlagert die Rechteerhöhung damit auf die Betriebssystemebene; dies setzt allerdings eine eigene Public-Key-Infrastruktur voraus. *HashiCorp Boundary* und *Teleport* schalten einen Sitzungs-Broker mit vollständiger Sitzungsaufzeichnung vor – sehr mächtig, aber mit erheblichem Betriebsaufwand verbunden. Identity-Provider wie *Keycloak* oder *Authentik* bieten Step-Up-Authentifizierung auf Basis von OpenID Connect, sind dabei jedoch auf den jeweiligen Identity-Provider zentriert.

Tabelle 1 ordnet diese Systeme ein. Das in dieser Arbeit entwickelte Modul verfolgt einen bewusst leichtgewichtigen Ansatz: Es führt die Rechteerhöhung nicht als externe Infrastruktur, sondern als eingebettete Bibliothek innerhalb der Anwendung durch. Dadurch entfällt zusätzlicher Betriebsaufwand, und das Modul bleibt durch das Ports-and-Adapters-Muster unabhängig vom konkreten Trägersystem.

Tabelle 1: Einordnung verwandter Systeme

System	Ansatz	Abgrenzung
HashiCorp Vault (SSH)	kurzlebige SSH-Zertifikate	OS-Ebene, eigene Infrastruktur
Boundary / Teleport	Session-Broker mit Recording	mächtig, hoher Betriebsaufwand
Keycloak / Authentik	OIDC-basierte Re-Auth	Identity-Provider-zentriert
Dieses Modul	eingebettete JIT-Erhöhung	leichtgewichtig, integrierbar

4 Konzept und Architektur

4.1 Überblick

Das Modul folgt dem Ports-and-Adapters-Muster: Der Kern kennt das Host-System nicht, sondern spricht ausschließlich gegen Interfaces. Der Host stellt konkrete Implementierungen bereit (Abbildung 1).

4.2 Lebenszyklus eines Grants

Eine Rechte-Erhöhung (Grant) durchläuft eine Zustandsmaschine (Abbildung 2). Alle Endzustände sind terminal – ein abgelaufener, widerrufener oder abgelehnter Grant wird nie wieder aktiv (fail-closed).

4.3 Ablauf

Der vollständige Ablauf umfasst vier Phasen (Abbildung 3). Phase 2 ist der Step-Up-Moment: ohne gültigen zweiten Faktor entsteht keine aktive Erhöhung.

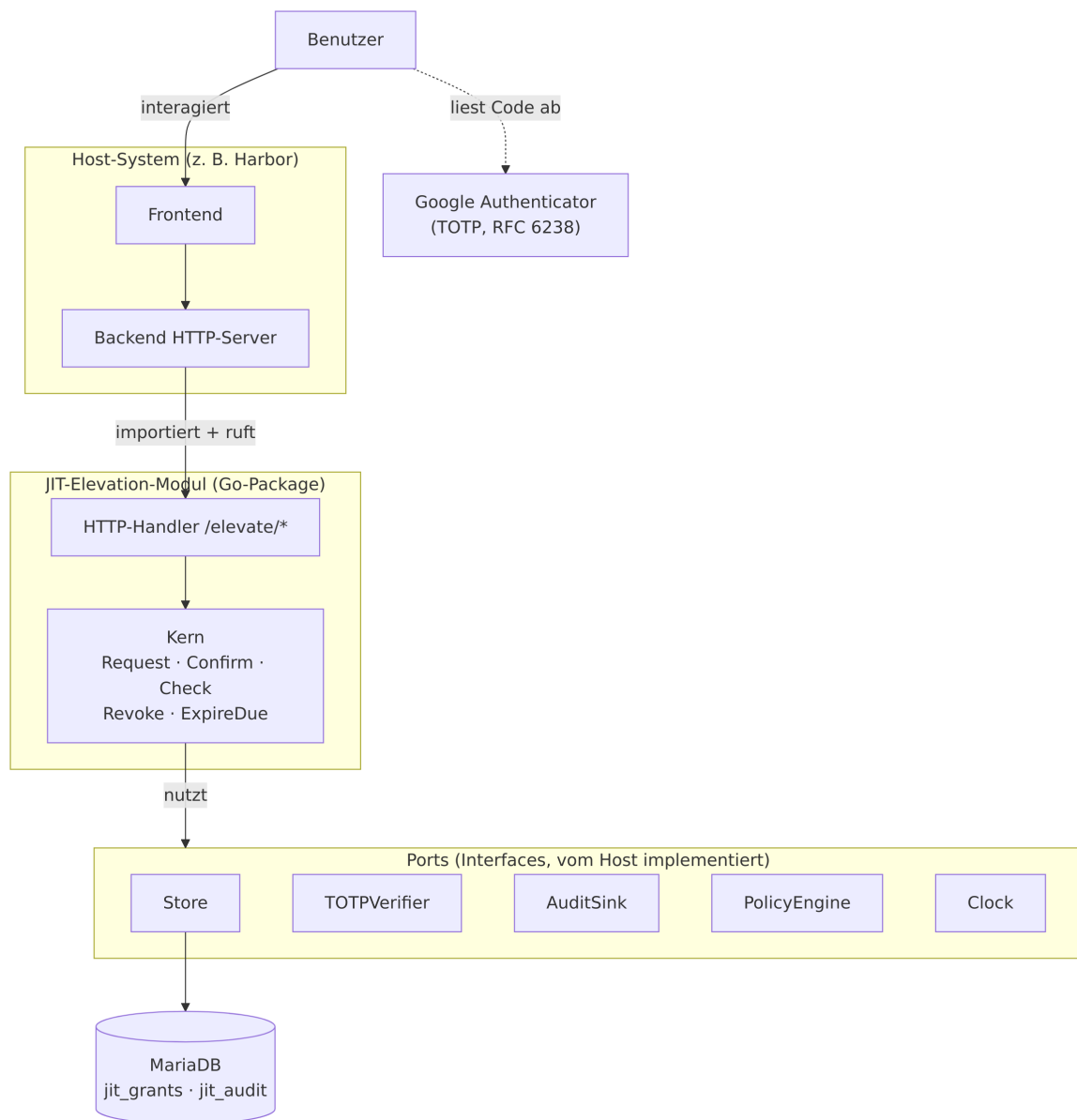


Abbildung 1: Architektur: Kern, Host und Adapter-Interfaces

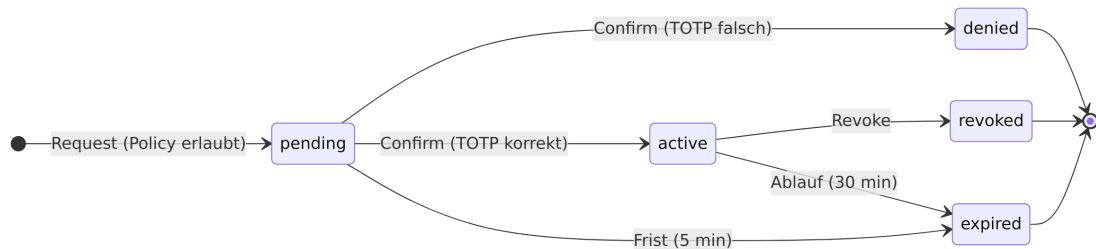


Abbildung 2: Zustandsmaschine eines Grants

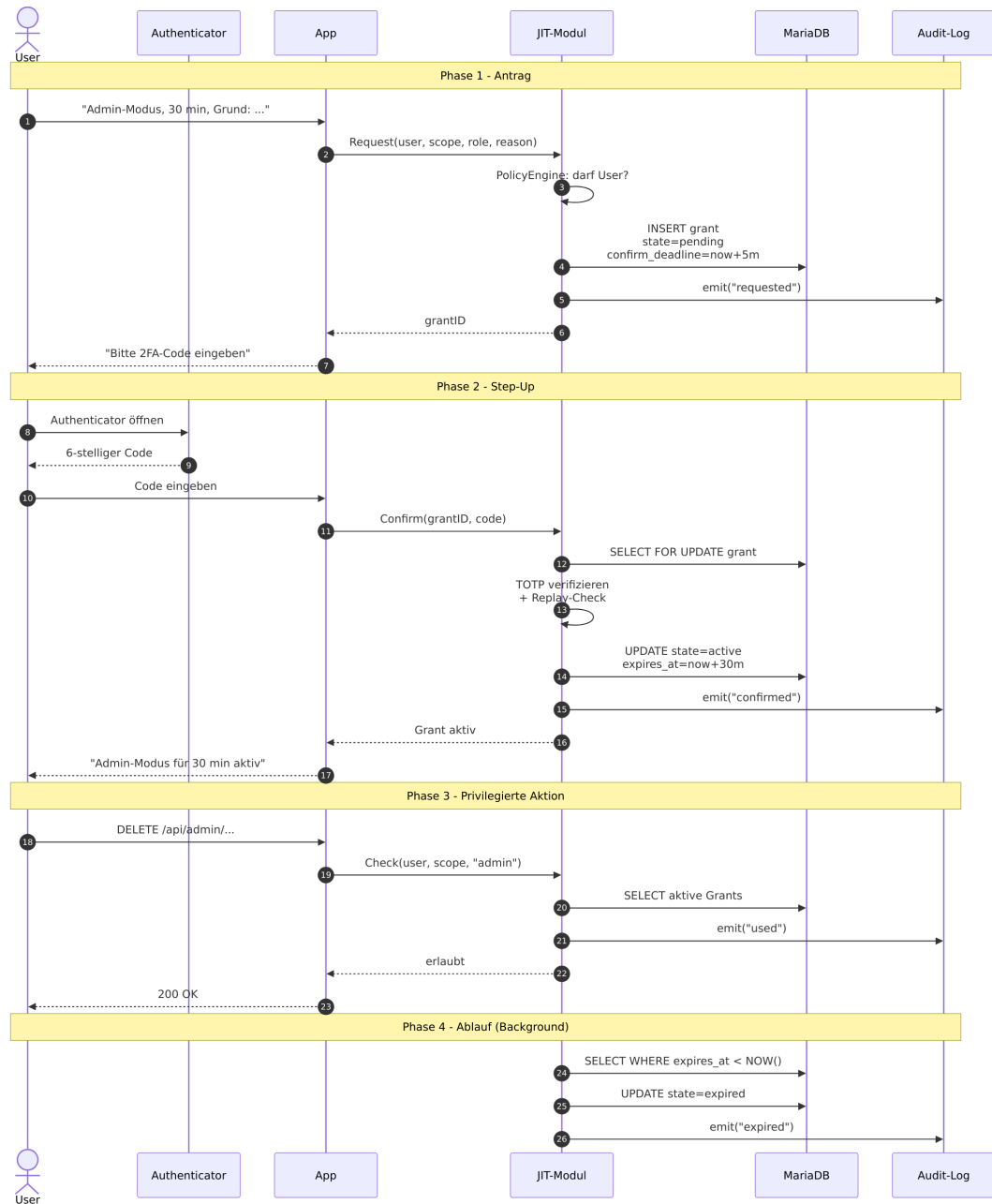


Abbildung 3: Ablauf: Antrag, Bestätigung, Nutzung, Ablauf

4.4 Datenmodell

Das Modul legt drei Tabellen an (Abbildung 4): die Grants selbst, das Audit-Log sowie eine Tabelle der bereits verbrauchten TOTP-Zeitfenster für den benutzerweiten Replay-Schutz. Die Audit-Tabelle bildet eine Hash-Kette: jede nachträgliche Manipulation eines Eintrags bricht die Kette am Folgeeintrag.

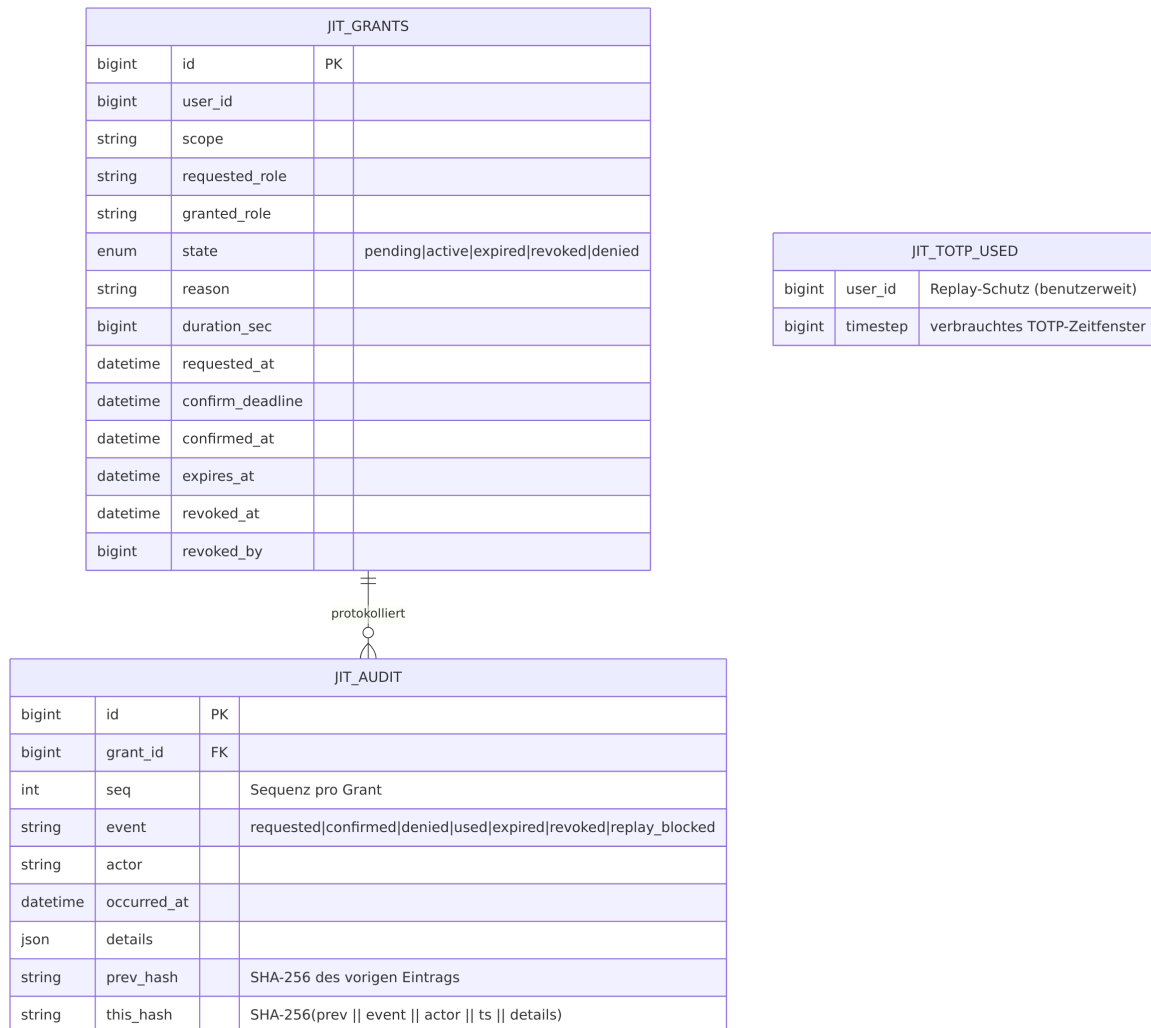


Abbildung 4: Datenmodell mit Audit-Hash-Kette

5 Sicherheitsanalyse

5.1 Vertrauensgrenzen

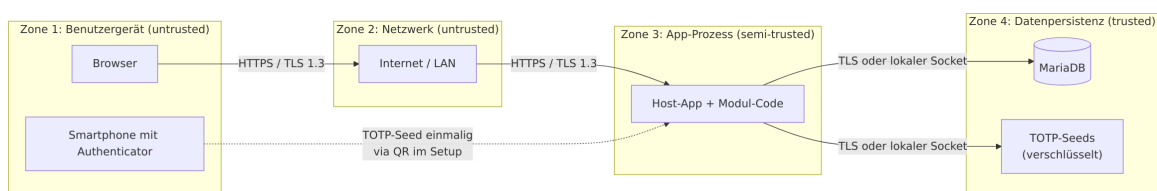


Abbildung 5: Vertrauensgrenzen als Grundlage der STRIDE-Analyse

Drei Grenzen müssen kryptografisch geschützt sein: Browser $\leftrightarrow$ Backend (TLS), Backend $\leftrightarrow$ Datenbank (TLS oder lokaler Socket) und der TOTP-Seed, der die Persistenz nie im Klartext verlässt.

5.2 Bedrohungsanalyse nach STRIDE

Die Analyse ordnet jeder Bedrohung eine konkrete Gegenmaßnahme im Entwurf bzw. im Code zu. Annahmen: Der Host authentifiziert die Primäridentität; TLS sichert den Transport; der TOTP-Seed liegt verschlüsselt im Secret-Store.

S – Spoofing. Erhöhung nur nach gültigem TOTP, den nur der echte Benutzer besitzt (Step-Up, **Confirm** Schritt 1). Restrisiko: kompromittierte Session *und* Seed – Minderung durch WebAuthn (Future Work).

T – Tampering. Audit-Manipulation wird durch die SHA-256-Hash-Kette erkennbar (`audit.go`); Grant-Daten sind nur über atomare Store-Methoden änderbar; TLS schützt Daten im Transit.

R – Repudiation. Lückenloses Protokoll aller Ereignisse (`requested`, `confirmed`, `used`, `revoked`, `expired`, `denied`, `replay_blocked`); auf kritischen Pfaden ist das Audit Pflicht – keine Erhöhung ohne Spur.

I – Information Disclosure. TOTP-Seeds werden vom Persistenz-Adapter verschlüsselt abgelegt (etwa mit AES-256-GCM), der Kern sieht sie nie; Grant-IDs sind 128-Bit-Zufallswerte (keine Enumeration); typisierte Fehler werden vom Host generisch nach außen abgebildet.

D – Denial of Service. Unbekannte Benutzer werden ohne Anlegen eines Grants abgewiesen; Rate-Limiting auf HTTP-Ebene. Restrisiko: Stören eines fremden pending-Antrags bei bekannter (nicht erratbarer) GrantID – Minderung durch Fehlversuchs-Zähler.

E – Elevation of Privilege. Kern-Bedrohungsklasse des Systems:

Tabelle 2: Rechteausweitung: Bedrohungen und Gegenmaßnahmen

Bedrohung	Gegenmaßnahme	Verortung
Höhere Rolle/Dauer als erlaubt	Policy deckelt Rolle + Dauer	<code>policy.go</code>
Erhöhung gilt nach Fristende	zeitlich erzwungen, Ticker	<code>ExpireDue</code>
Doppel-Aktivierung (Race)	atomar <code>state=pending</code>	<code>ActivateGrant</code>
TOTP-Replay	test-and-set pro Zeitfenster	<code>Confirm</code>
Umgehung Frontend-Hiding	serverseitig, fail-closed	<code>Check</code>

5.3 Fail-closed als durchgängiges Prinzip

Jeder Fehlerpfad – Policy-Fehler, fehlender/falscher TOTP, Replay, abgelaufene Frist, Store-Fehler – führt zu keinem aktiven Grant. **Check** liefert im Zweifel **false**. Dies ist durch Tests belegt (u. a. `TestCheck_FailsClosedOnStoreError`).

6 Implementierung

6.1 Modul-Struktur

Das Modul ist in Go geschrieben (rund 1360 Zeilen inklusive Tests); der Kern hat keine externen Abhängigkeiten.

jitelevation/	
doc.go, ids.go, grant.go, errors.go	Domaenen-Typen + State-Machine
ports.go	5 Interfaces (Store, TOTPVerifier, ...)
audit.go	AuditEvent + SHA-256-Hash-Kette
policy.go	Beispiel-Policy mit Rollen-Hierarchie
module.go	Kern: Request/Confirm/Check/Revoke/ExpireDue
adapter/memory/	In-Memory-Store (Tests/Demo)
adapter/totp/	TOTP-Verifier nach RFC 6238

6.2 Die Schnittstellen (Ports)

```

1 type Store interface {
2     CreateGrant(ctx, g Grant) error
3     GetGrant(ctx, id GrantID) (Grant, error)
4     ActivateGrant(ctx, id GrantID, confirmedAt, expiresAt time.Time) error
5     DenyGrant(ctx, id GrantID, at time.Time) error
6     RevokeGrant(ctx, id GrantID, by UserID, at time.Time) error
7     ActiveGrants(ctx, user UserID, scope string) ([]Grant, error)
8     ExpireDue(ctx, now time.Time) ([]GrantID, error)
9     MarkTOTPUsed(ctx, user UserID, timestep uint64) (firstUse bool, err error)
10 }
```

6.3 Der sicherheitskritische Pfad: Confirm

Die Bestätigung ist der Step-Up-Schritt. Jeder Fehler führt zu keiner Erhöhung.

```

1 // 1) Zweitfaktor pruefen. Ein falscher Code verbrennt den Antrag.
2 timestep, err := m.totp.Verify(ctx, g.User, totpCode)
3 if errors.Is(err, ErrInvalidTOTP) {
4     m.denyBestEffort(ctx, id, g.User, now, "totp_falsch")
5     return Grant{}, ErrInvalidTOTP
6 }
7 // 2) Replay-Schutz: Zeitfenster atomar als verbraucht markieren.
8 firstUse, err := m.store.MarkTOTPUsed(ctx, g.User, timestep)
9 if !firstUse { /* ... ErrTOTPReplay ... */ }
10 // 3) Atomar aktivieren (gewinnt nur ein paralleler Aufruf).
11 if err := m.store.ActivateGrant(ctx, id, now, now.Add(g.Duration)); err != nil {
12     return Grant{}, err
13 }
```

6.4 TOTP nach RFC 6238

Der Zweitfaktor-Verifier implementiert RFC 6238 (TOTP) auf Basis von RFC 4226 (HOTP) mit der Standardbibliothek. Die kryptografische Primitive (HMAC-SHA1) stammt aus `crypto/hmac`; der Code-Vergleich erfolgt zeitkonstant (`crypto/subtle`) gegen Timing-Angriffe. Die Korrektheit ist gegen die offiziellen Testvektoren beider RFCs nachgewiesen. Produktiv empfiehlt sich eine auditierte Bibliothek (z. B. `pquerna/otp`).

7 Evaluation

7.1 Tests und Abdeckung

15 automatisierte Tests, ausgeführt mit aktiviertem Race-Detector (`go test -race`). Testabdeckung: Kern 84 %, TOTP-Adapter 91 %. Abgedeckt sind u. a. Happy-Path, falscher Zweitfaktor, Replay, Fristablauf, Widerruf, Policy-Ablehnung, fail-closed bei Store-Fehler, Nebenläufigkeit und die Integrität der Audit-Hash-Kette.

7.2 Durch Tests aufgedeckte Defekte

Der Nebenläufigkeitstest deckte zwei Defekte auf, die im Einzeltest unsichtbar blieben:

1. **Replay-Sabotage:** Bei parallelen Bestätigungen desselben Antrags setzte der Replay-Verlierer den Grant auf `denied`, bevor der Gewinner aktivieren konnte. Behoben, indem ein Replay den Antrag nicht mehr verwirft, sondern als eigenes Sicherheitsereignis (`replay_blocked`) protokolliert wird.
2. **Data-Race im Audit-Sink:** Der Race-Detector belegte, dass `AuditSink.Emit` nebenläufig aufgerufen wird; die Anforderung der Nebenläufigkeitssicherheit wurde am Interface dokumentiert.

7.3 Limitierungen

- DoS auf pending-Anträge bei bekannter GrantID (Minderung: Fehlversuchs-Zähler).
- TOTP ist nicht phishing-resistent (Minderung: WebAuthn/FIDO2).
- Das Modul vertraut der Host-Authentifizierung (bewusste Scope-Grenze).
- Audit-Schreibfehler auf Nebenpfaden werden best-effort behandelt; produktiv koppelt der Store-Adapter Zustandsänderung und Audit in einer Transaktion.

8 Fazit und Ausblick

Die Arbeit hat gezeigt, dass sich Just-in-Time-Privilege-Elevation als schlankes, wiederverwendbares Modul umsetzen lässt, ohne eigene Kryptografie zu entwickeln. Stattdessen werden bewährte Bausteine kombiniert: zeitbasierte Einmalpasswörter (TOTP) für die Step-Up-Bestätigung und eine SHA-256-Hash-Kette für ein manipulationssicheres Protokoll. Die

verschlüsselte Ablage der Seeds (etwa mit AES-256-GCM) ist dabei Aufgabe des jeweiligen Persistenz-Adapters und im vorgestellten Kern bewusst nicht enthalten. Die durchgängige Fail-closed-Auslegung sowie die Entkopplung über klar definierte Schnittstellen wurden durch automatisierte Tests einschließlich Nebenläufigkeitsprüfungen belegt; zwei dabei aufgedeckte Defekte konnten behoben werden.

Für einen produktiven Einsatz bleiben mehrere Erweiterungen offen. Die Rechteerhöhung ließe sich von der Anwendungs- auf die Betriebssystemebene ausweiten, etwa als zeitlich befristetes `sudo`. Ein Vier-Augen-Prinzip könnte sicherheitskritische Erhöhungen an die Freigabe einer zweiten Person binden. Der nicht phishing-resistente TOTP-Faktor ließe sich durch WebAuthn bzw. FIDO2 ersetzen. Schließlich steht die produktive Implementierung der Persistenz-Schnittstelle – etwa auf Basis von MariaDB – aus, die im Rahmen dieser Arbeit durch einen In-Memory-Adapter ersetzt wurde.